

▼ Recitation 3

This notebook will use the linear regression model to predict the housing price in Philadelphia based on a dataset from Kaggle, which can be found [here](#)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
import statsmodels.formula.api as smf
import statsmodels.api as sm
import matplotlib.cm as cm
import matplotlib.colors as colors
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score
import folium
from geopy.geocoders import Nominatim
import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
```

▼ Part 1 Data Preprocessing

```
df = pd.read_csv('/content/Properties_philly_Kraggle_v2.csv')
```

```
df.columns
```

```
Index(['Address', 'Zillow Address', 'Sale Date', 'Opening Bid',
      'Sale Price/bid price', 'Book/Writ', 'OPA', 'Postal Code', 'Attorney',
      'Ward', 'Seller', 'Buyer', 'Sheriff Cost', 'Advertising', 'Other',
      'Record Deed', 'Water', 'PGW', ' Avg Walk&Transit score ',
      ' Violent Crime Rate ', ' School Score ', 'Zillow Estimate',
      'Rent Estimate', 'taxAssessment', 'yearBuilt', 'finished \n(SqFt)',
      ' bathrooms ', ' bedrooms ', 'PropType', 'Average comps'],
      dtype='object')
```

▼ Rename columns

Renaming the column names of a dataframe is a good habit. It's not only for the later convenient use, but python is picky for the format of column names. So we can use [rename](#) to replace the space and special symbols with underlines on column names.

```
# Rename some columns
df = df.rename(columns = {'Sale Price/bid price':'Price',
                          'Sheriff Cost': 'Sheriff_Cost',
                          ' Avg Walk&Transit score ':'Avg_Transit_Score',
                          ' Violent Crime Rate ':'Violent_Crime_Rate',
                          ' School Score ':'School_Score',
                          'finished \n(SqFt)':'SqFt',
                          ' bathrooms ':'Bathrooms',
                          ' bedrooms ':'Bedrooms',
                          'Average comps':'Avg_Comps'})
```

▼ Drop Missing Values

As data comes in many shapes and forms, pandas aims to be flexible with regard to handling missing data. What we do here is to use [dropna](#) to drop the row if all values are NA in that row.

```
# Drop the row if all values are NA in that row
df.dropna(how = 'all',inplace = True)
df.reset_index(inplace = True,drop = True)
```

▼ Apply and Lambda Usage in Pandas

After observing the "yearBuilt" column, we noticed that the format of the values in this column is like "2020". Since we definitely want to take this column into consideration for our model, then we can convert this column to the number of years built. (2021-2012 = 9). The function we are using is [apply](#). You can also refer to this [link](#) to know more about the apply and lambda usage in pandas.

```
# Convert the column yearBuilt to year. e.g. 2012 converted to 2021-2012 = 9
df['years'] = df['yearBuilt'].apply(lambda x:2021-x)
df.drop('yearBuilt',axis=1,inplace = True)
```

For "Price" column, we need to remove special symbols like "\$", comma, and space and convert the type from string to float. So does column "Avg_Comps"

```
# Convert price column from string type to float type
df['Price'] = df['Price'].apply(lambda x:x.replace('$',''))
df['Price'] = df['Price'].apply(lambda x:x.replace(' ',''))
df['Price'] = df['Price'].apply(lambda x:float(x.replace(',','')))
```

```
# Average comparables column
df['Avg_Comps'] = df['Avg_Comps'].apply(lambda x:float(x.replace(',','')))
```

▼ Convert a Categorical Column into Dummy Columns

For column "PropType", we use [get_dummies](#) to convert this categorical variable into dummy/indicator variables. Then use [concat](#) to concatenate pandas objects along a particular axis with optional set logic along the other axes.

```
# One hot encoding
df = pd.concat([df, pd.get_dummies(df['PropType'], prefix='Type')], axis = 1)
df.drop('PropType', axis = 1, inplace = True)
```

▼ Imputation

Imputation is not necessarily in this course. When should we use imputation to deal with missing values?

```
# Regression Imputation on Bathroom column
df['Bathrooms'] = df['Bathrooms'].apply(lambda x: eval(x.replace(' - ', 'None'))))

missing_bathroom_index = df[df['Bathrooms'].isna()].index

known_bathrooms = df[~df['Bathrooms'].isna()]
unknown_bathrooms = df[df['Bathrooms'].isna()]

known_bathrooms_x = known_bathrooms[['SqFt', 'Type_Condominium', 'Type_MultiFamily2To4',
                                     'Type_SingleFamily', 'Type_Townhouse', 'years']]
known_bathrooms_y = known_bathrooms['Bathrooms']

unknown_bathrooms_x = unknown_bathrooms[['SqFt', 'Type_Condominium', 'Type_MultiFamily2To4',
                                          'Type_SingleFamily', 'Type_Townhouse', 'years']]

imputation_model = LinearRegression()
imputation_model.fit(known_bathrooms_x, known_bathrooms_y)
for i in missing_bathroom_index:
    df.at[i, 'Bathrooms'] = round(imputation_model.predict(unknown_bathrooms_x.loc[[i]])[0])

# Regression Imputation on Bedroom column
df['Bedrooms'] = df['Bedrooms'].apply(lambda x: eval(x.replace(' - ', 'None'))))

missing_bedroom_index = df[df['Bedrooms'].isna()].index

known_bedrooms = df[~df['Bedrooms'].isna()]
unknown_bedrooms = df[df['Bedrooms'].isna()]

known_bedrooms_x = known_bedrooms[['SqFt', 'Type_Condominium', 'Type_MultiFamily2To4',
                                    'Type_SingleFamily', 'Type_Townhouse', 'years']]
known_bedrooms_y = known_bedrooms['Bedrooms']
```

```
unknown_bedrooms_x = unknown_bedrooms[['SqFt', 'Type_Condominium', 'Type_MultiFamily2To4',
                                         'Type_SingleFamily', 'Type_Townhouse', 'years']]
```

```
imputation_model = LinearRegression()
imputation_model.fit(known_bedrooms_x, known_bedrooms_y)
for i in missing_bedroom_index:
    df.at[i, 'Bedrooms'] = round(imputation_model.predict(unknown_bedrooms_x.loc[[i]])[0])
```

```
df.describe()
```

SqFt	Bathrooms	Bedrooms	Avg_Comps	years	Type_Condominium	Type_M
615.000000	615.000000	615.000000	615.000000	615.000000	615.000000	
306.229268	1.378862	3.131707	114355.620780	127.708943	0.294309	
440.709941	0.683359	0.663536	75008.243672	279.049115	0.456102	
0.000000	1.000000	1.000000	21166.670000	15.000000	0.000000	
056.000000	1.000000	3.000000	63665.155000	76.000000	0.000000	
200.000000	1.000000	3.000000	94285.710000	96.000000	0.000000	
440.000000	2.000000	3.000000	144842.525000	97.000000	1.000000	
564.000000	6.000000	6.000000	790500.000000	2021.000000	1.000000	

```
df = df[df['years'] != 2021]
df.reset_index(inplace = True, drop = True)
```

```
df.describe()
```

SqFt	Bathrooms	Bedrooms	Avg_Comps	years	Type_Condominium	Type_M
602.000000	602.000000	602.000000	602.000000	602.000000	602.000000	
308.906977	1.380399	3.127907	114650.340432	86.823920	0.300664	
441.008066	0.684860	0.661707	75426.446629	18.502387	0.458928	
0.000000	1.000000	1.000000	21166.670000	15.000000	0.000000	
056.000000	1.000000	3.000000	63664.397500	75.250000	0.000000	
200.000000	1.000000	3.000000	94390.155000	96.000000	0.000000	
440.000000	2.000000	3.000000	144921.262500	96.000000	1.000000	
564.000000	6.000000	6.000000	790500.000000	146.000000	1.000000	

▼ Convert Address to Latitudes and Longitudes

Philly coordinates (copy paste from online resources)

```

philly_dict = {"type": "GeometryCollection", "geometries": [{"type": "MultiPolygon", "coordinates": philly_coordinates}]}
philly_coordinates = philly_dict['geometries'][0]['coordinates'][0][0]
philly_coordinates = [[i[1], i[0]] for i in philly_coordinates]

# Define a function to retrieve latitudes and longitudes for specific locations
def geocodeN(address):
    gps=Nominatim(user_agent="my-application")
    location=gps.geocode(address+', Philadelphia, PA', timeout=20)
    if not location:
        return 0,0
    return location.longitude, location.latitude

# Find Longitude and Latitudes
df['Longitude'] = df['Address'].apply(lambda x: geocodeN(x)[0])
df['Latitude'] = df['Address'].apply(lambda x: geocodeN(x)[1])

```

▼ Part 2 Data Modeling

Pick up some columns we think would have impact on the housing prices.

```

df_linear = df[['Price', 'Sheriff_Cost', 'Advertising', 'Water', 'PGW', 'Avg_Transit_Score',
                'Violent_Crime_Rate', 'School_Score', 'SqFt', 'Bathrooms', 'years',
                'Bedrooms', 'Avg_Comps', 'Type_Condominium', 'Type_MultiFamily2To4', 'Type_SingleFamily']]

df_linear.describe()

```

Rate	School_Score	SqFt	Bathrooms	years	Bedrooms	Avg_Comps	Type
3000	615.000000	615.000000	615.000000	615.000000	615.000000	615.000000	
3829	13.654358	1306.229268	1.378862	127.708943	3.131707	114355.620780	
7038	11.490496	440.709941	0.683359	279.049115	0.663536	75008.243672	
3000	3.340000	0.000000	1.000000	15.000000	1.000000	21166.670000	
3000	7.280000	1056.000000	1.000000	76.000000	3.000000	63665.155000	
3000	10.870000	1200.000000	1.000000	96.000000	3.000000	94285.710000	
3000	14.200000	1440.000000	2.000000	97.000000	3.000000	144842.525000	
3000	69.500000	4564.000000	6.000000	2021.000000	6.000000	790500.000000	

Use [train_test_split](#) to split our dataset into two parts: train dataset and test dataset.

```
x = df_linear.drop('Price',axis = 1)
y = df_linear['Price']
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.2)
```

Create linear regression model

```
model = smf.ols('Price~Sheriff_Cost+Advertising+Water+PGW+Avg_Transit_Score\
+Violent_Crime_Rate+School_Score+SqFt+Bathrooms+years\
+Bedrooms+Avg_Comps+Type_Condominium+Type_MultiFamily2To4+Type_SingleFamily+Type_Tc
model.summary()
```

OLS Regression Results						
Dep. Variable:	Price	R-squared:	0.967			
Model:	OLS	Adj. R-squared:	0.966			
Method:	Least Squares	F-statistic:	1157.			
Date:	Fri, 12 Feb 2021	Prob (F-statistic):	0.00			
Time:	04:09:04	Log-Likelihood:	-6334.8			
No. Observations:	602	AIC:	1.270e+04			
Df Residuals:	586	BIC:	1.277e+04			
Df Model:	15					
Covariance Type: nonrobust						
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-2.303e+04	5202.668	-4.427	0.000	-3.32e+04	-1.28e+04
Sheriff_Cost	33.8492	0.477	70.939	0.000	32.912	34.786
Advertising	-10.5276	1.655	-6.363	0.000	-13.777	-7.278
Water	0.2358	0.095	2.473	0.014	0.049	0.423
PGW	0.1684	0.156	1.080	0.281	-0.138	0.475
Avg_Transit_Score	98.0754	65.075	1.507	0.132	-29.734	225.884
Violent_Crime_Rate	1970.8010	1241.134	1.588	0.113	-466.812	4408.414
School_Score	144.7504	50.285	2.879	0.004	45.989	243.512
SqFt	0.3308	1.324	0.250	0.803	-2.270	2.932
Bathrooms	-119.5355	692.457	-0.173	0.863	-1479.535	1240.464
years	-88.0784	23.708	-3.715	0.000	-134.641	-41.516
Bedrooms	-233.5647	758.225	-0.308	0.758	-1722.734	1255.605
Avg_Comps	0.0741	0.009	8.244	0.000	0.056	0.092
Type_Condominium	-4969.0466	1587.810	-3.129	0.002	-8087.538	-1850.555
Type_MultiFamily2To4	-6133.1075	3727.014	-1.646	0.100	-1.35e+04	1186.823
Type_SingleFamily	-5684.5695	1560.258	-3.643	0.000	-8748.948	-2620.191
Type_Townhouse	-6245.0653	1588.718	-3.931	0.000	-9365.339	-3124.791
Omnibus:	167.780	Durbin-Watson:	1.705			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	9835.452			
Skew:	-0.204	Prob(JB):	0.00			
Kurtosis:	22.798	Cond. No.	1.05e+21			

Warnings:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 1.04e-29. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

▼ Part 3 Data Visualization

In order to distinguish the houses by their price, we use [sort_values](#) to split dataframes into three categories.

```
df_sort = df.sort_values(by = 'Price', ascending = False)
df_top_200 = df_sort.iloc[:200]
df_top_400 = df_sort.iloc[200:400]
df_lowest = df_sort.iloc[400:]
```

▼ Map Visualization

We use Folium to mark every house on our map.

```
# House location

latitude = 40
longitude = -75.1

# Instantiate a feature group for the incidents in the dataframe
incidents = folium.map.FeatureGroup()

for lat, lng, in zip(df.Latitude, df.Longitude):
    incidents.add_child(
        folium.CircleMarker(
            [lat, lng],
            radius=7,
            color='yellow',
            fill=True,
            fill_color='red',
            fill_opacity=0.4
        )
    )

# Add incidents to map
philly_map = folium.Map(location=[latitude, longitude], zoom_start=13)
philly_map.add_child(incidents)

for lat, lng, label in zip(list(df_top_200.Latitude), list(df_top_200.Longitude), list(df_top_200.Price)):
    philly_map.add_child(folium.Marker([lat, lng], popup=label, icon=folium.Icon(color='darkred')).add_to(philly_map))

for lat, lng, label in zip(list(df_top_400.Latitude), list(df_top_400.Longitude), list(df_top_400.Price)):
    philly_map.add_child(folium.Marker([lat, lng], popup=label, icon=folium.Icon(color='red')).add_to(philly_map))

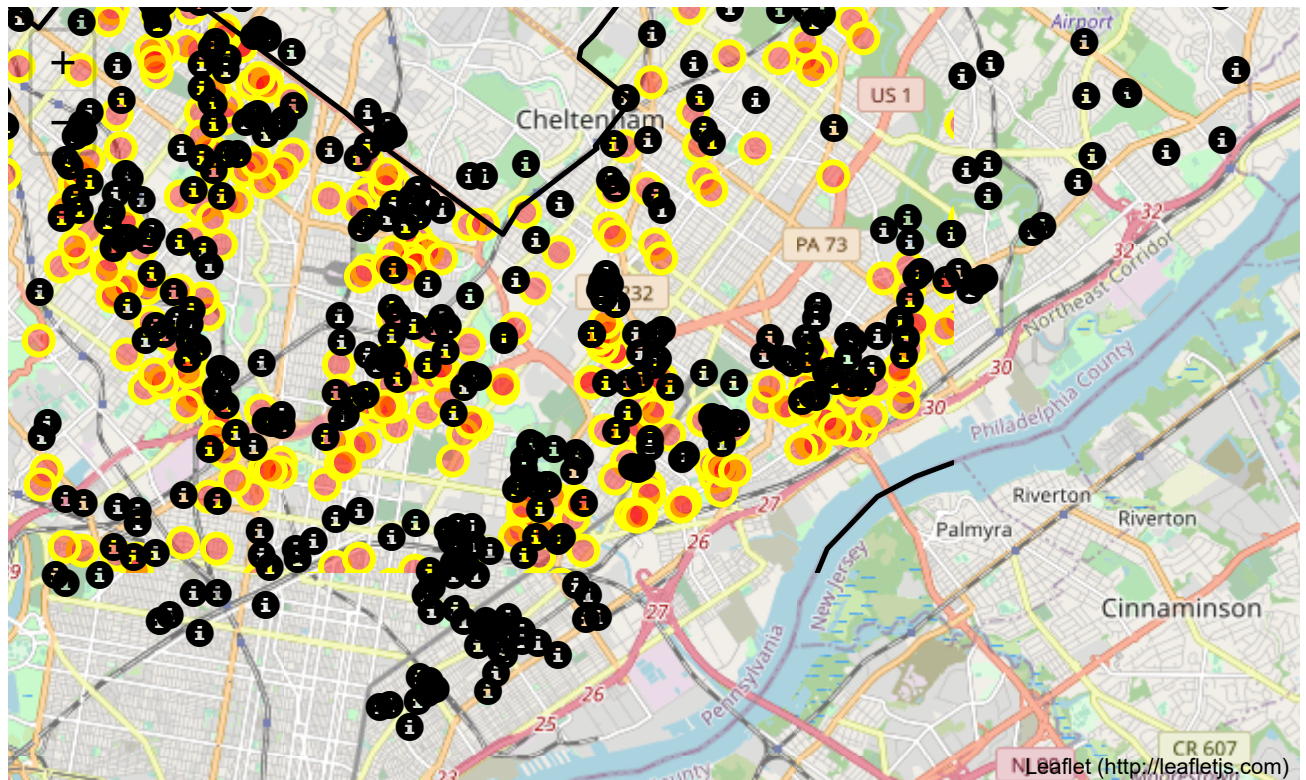
for lat, lng, label in zip(list(df_lowest.Latitude), list(df_lowest.Longitude), list(df_lowest.Price)):
    philly_map.add_child(folium.Marker([lat, lng], popup=label, icon=folium.Icon(color='lightred')).add_to(philly_map))
```

```

folium.PolyLine(locations=philly_coordinates,color="black", weight=2.5, opacity=1).add_to(

# add incidents to map
philly_map.add_child(incidents)

```



▼ Parse Data from Websites

We may also want to check whether our model makes sense or not. For example, the t-statistic of School_Score is greater than 2 so we might want to know whether the expensive houses are closer to the school regions. Unfortunately, we don't have school address information.

But don't worry! We can parse all of the school information from the website and then mark them on the map as well.

```

import requests
from lxml import html

```

```

w = requests.get("https://en.wikipedia.org/wiki/List_of_schools_of_the_School_District_of")
dom_tree = html.fromstring(w.content)

```



```

<div class="mw-parser-output">
  <div class="thumb tright">...</div>
  <p>...</p>
  <table class="wikitable">...</table>
  <p>...</p>
  <p>...</p>
  <div id="toc" class="toc" role="navigation" aria-
labelledby="mw-toc-heading">...</div>
  <h2>...</h2>
  <div class="thumb tright">...</div>
  <div class="thumb tright">...</div>
  <div class="thumb tright">...</div>
  <div class="thumb tright">...</div>
  <ul>
    <li>...</li>
    <li>...</li>
    <li>...</li>
    <li>...</li>
  </ul>

```

```

school1 = pd.Series(dom_tree.xpath('//div[contains(@class,"mw-parser")]//ul/li/text()').t
school2 = pd.Series(dom_tree.xpath('//div[contains(@class,"mw-parser")]//ul/li/a/text()'))
school = pd.concat([school1,school2],axis = 0)

```

The data parsed from wikipedia would also contain some useless information and symbols that we don't need. So we also do some data cleaning here.

```

school['School'] = school['School'].apply(lambda x:x.replace("\n",""))
school['School'] = school['School'].apply(lambda x:x.replace(",",""))
school['School'] = school['School'].apply(lambda x:str(0) if "(" in x else x)
school['School'] = school['School'].apply(lambda x:x if "School" in x else None)
school = school[~school['School'].isna()]
school.reset_index(inplace = True,drop = True)

```

```

school['Location'] = school['School'].apply(lambda x:geocodeN(x))
school['Longitude'] = school['Location'].apply(lambda x:x[0])
school['Latitude'] = school['Location'].apply(lambda x:x[1])
school = school[school['Location'] != (0,0)]

```

School location

```

latitude = 40
longitude = -75.1

```

Instantiate a feature group for the incidents in the dataframe

```

incidents = folium.map.FeatureGroup()

# Add incidents to map
philly_map = folium.Map(location=[latitude, longitude], zoom_start=13)
philly_map.add_child(incidents)

for lat, lng, label in zip(list(df_top_200.Latitude), list(df_top_200.Longitude), list(df_top_200.Label)):
    folium.Marker([lat, lng], popup=label, icon=folium.Icon(color='darkred')).add_to(philly_map)

for lat, lng, label in zip(list(df_top_400.Latitude), list(df_top_400.Longitude), list(df_top_400.Label)):
    folium.Marker([lat, lng], popup=label, icon=folium.Icon(color='red')).add_to(philly_map)

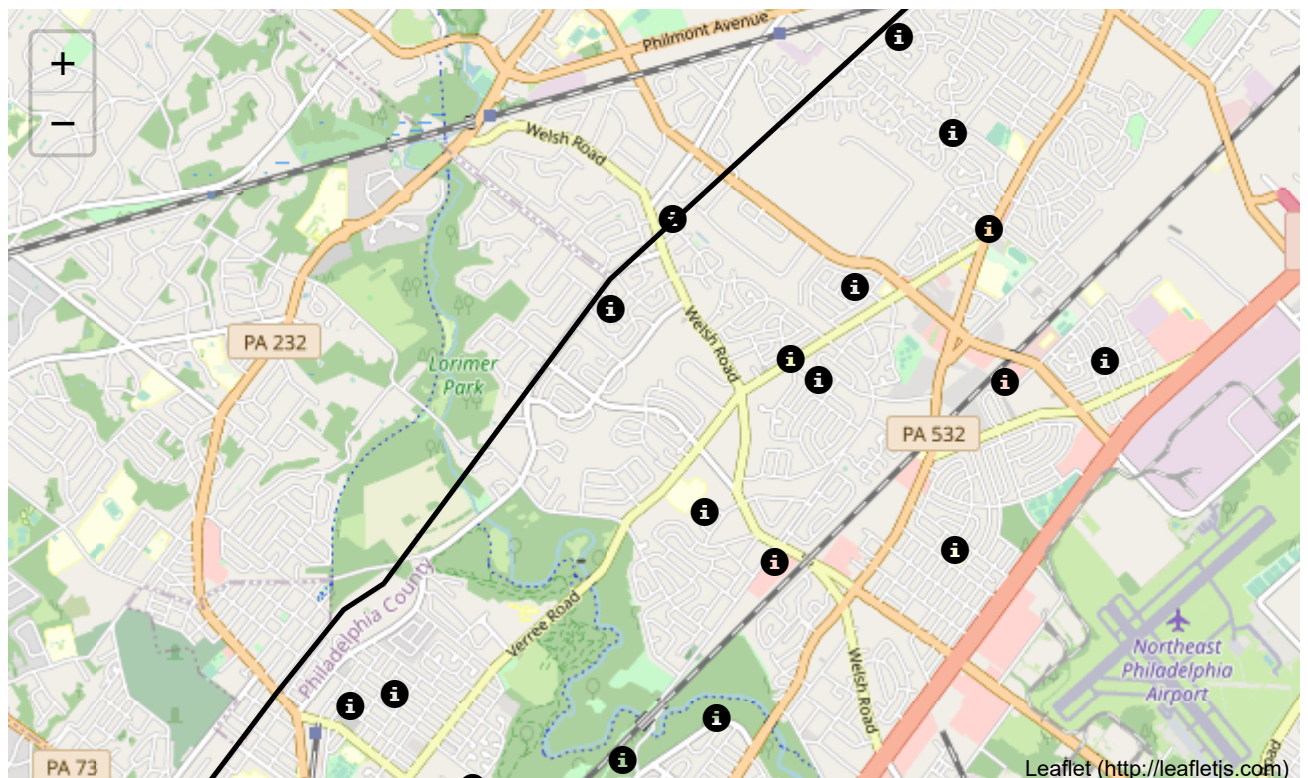
for lat, lng, label in zip(list(df_lowest.Latitude), list(df_lowest.Longitude), list(df_lowest.Label)):
    folium.Marker([lat, lng], popup=label, icon=folium.Icon(color='lightred')).add_to(philly_map)

for lat, lng, label in zip(list(school.Latitude), list(school.Longitude), list(school.School)):
    folium.Marker([lat, lng], popup=label, icon=folium.Icon(color='black')).add_to(philly_map)

folium.PolyLine(locations=philly_coordinates,color="black", weight=2.5, opacity=1).add_to(philly_map)

# add incidents to map
philly_map.add_child(incidents)

```



The black markers(schools) on the map are surrounded by dark red markers(expensive houses). The result is aligned with what we conclude from our model: the variable `School_Score` has a positive relationship with housing prices.